ARQUITECTURA CLIENTE-SERVIDOR

PATRONES DE DESARROLLO WEB

9 DE NOVIEMBRE DE 2026
JOAQUIN VALENTIN OTTONELLO
DAW2 · DESARROLLO WEB EN ENTORNO CLIENTE

INDICE

INTRODUCCIÓN....2
1. DESARROLLO....3
1.1. Concepto general....3
1.2. Componentes principales....3
1.3. Comunicación entre cliente y servidor....5
1.4. Ejemplo práctico: consulta de un producto en una tienda online....6
1.5. Modelo de capas....7
1.6. Modelo-vista-controlador....8
1.7. API REST....9
1.8. SPA (Single Page Application)....10
1.9. SSR (Server Side Rendering)....11
1.10. Arquitectura Jamstack....12
1.11. Frontend....12
1.12. Backend....13
2. CASO DE ESTUDIO: NETFLIX....13
2.1. Arquitectura cliente-servidor....13
2.2. Posible uso de MVC....14
2.3. APIs utilizadas....14
2.4. Frontend y backend....15
2.5. Uso de microservicios....15
2.6. Tecnologías que podrían emplearse....15
3. DIAGRAMA DE UNA APLICACIÓN WEB PROPIA: BIBLIOTECA DECROLY....16
3.1. Componentes....16
3.2. Patrón arquitectónico utilizado....17
3.3. Justificación de las decisiones tomadas....17
CONCLUSIONES....18
BIBLIOGRAFÍA....19

INTRODUCCIÓN

En este documento se va a explicar cómo funciona por dentro una aplicación web, empezando por la base de todo, que es la arquitectura cliente-servidor, y siguiendo con los patrones y enfoques que se usan hoy en día para organizar una aplicación: el modelo de capas, MVC, las API REST, las SPA, el renderizado en servidor (SSR) y Jamstack (patrón de desarrollo web, donde las páginas se generan antes de que nadie las pida, de manera que se cumple el prerrenderizado; y, a su vez, la lógica, tanto front como back, va separada y no depende una de la otra, cumpliendo así con el desacoplamiento). La idea no es quedarse en la definición de cada cosa, sino entender qué problema resuelve cada una y cuándo tiene sentido usarla, que al final es lo que uno tiene que decidir cuando empieza un proyecto.

Para que no se quede todo en teoría, el trabajo incluye diagramas hechos por mí, como el recorrido de una petición en una tienda online o la diferencia entre una SPA y una página renderizada en servidor.

Después se analiza un caso real, Netflix, y por último se diseña el diagrama de una aplicación propia, la Biblioteca Decroly (API REST con Spring Boot, desarrollada en el módulo DWES, Desarrollo Web en Entorno Servidor), justificando las decisiones tomadas y planteando cómo se podría ampliar con un frontend y una base de datos adecuados.

1. DESARROLLO

1.1. Concepto general

¿Qué es una arquitectura cliente-servidor?

La arquitectura cliente-servidor es un modelo en el que una aplicación se reparte entre dos partes que se comunican por red: el **cliente**, que es quien pide algo, y el **servidor**, que es quien tiene los datos o el servicio y responde. El cliente no necesita saber cómo está hecho el servidor por dentro, solo cómo pedirle las cosas; y el servidor no necesita saber qué hace el cliente con lo que le devuelve. Esa separación de responsabilidades es precisamente la primera restricción sobre la que Roy Fielding construyó más tarde el estilo REST (Fielding, 2000).

Un ejemplo del día a día: al abrir Gmail en el navegador, el navegador (cliente) envía peticiones a los servidores de Google, que son los que guardan los correos, comprueban la sesión y devuelven la bandeja de entrada. Si se entra después desde el móvil, los correos siguen ahí, porque nunca estuvieron en el equipo del usuario.

Diferencias respecto a una aplicación monolítica local

Una aplicación monolítica local es la que se instala y se ejecuta entera en el equipo del usuario: la interfaz, la lógica y los datos viven en la misma máquina, y normalmente en el mismo ejecutable. Sería el caso de un programa de facturación de los de antes, que guarda todo en un fichero o en una base de datos Access dentro del propio PC. Las diferencias principales se resumen en la siguiente tabla:

Tabla 1. Aplicación monolítica local frente a arquitectura cliente-servidor

Aspecto	Aplicación monolítica local	Arquitectura cliente-servidor
Dónde se ejecuta	Todo en el equipo del usuario	Repartida entre cliente y servidor
Datos	Guardados en el propio equipo	Centralizados en el servidor
Acceso	Solo desde ese equipo	Desde cualquier dispositivo con conexión
Actualizaciones	Hay que instalarlas en cada equipo	Se despliegan una vez en el servidor
Escalabilidad	Limitada al hardware del equipo	Se pueden añadir o mejorar servidores

Ventajas e inconvenientes

Tabla 2. Ventajas e inconvenientes de la arquitectura cliente-servidor

Ventajas	Inconvenientes
Datos centralizados: una única fuente de verdad, sin versiones distintas del mismo dato en cada equipo.	**Dependencia de la red:** sin conexión, o si el servidor se cae, el cliente se queda sin servicio.
Mantenimiento: un cambio en el servidor llega a todos sin que tengan que instalar nada.	**Punto único de fallo:** sin redundancia, una caída del servidor afecta a todos los usuarios.
Escalabilidad: si la carga crece, se añaden o mejoran servidores sin tocar a los clientes.	**Latencia:** cada petición tiene que ir y volver, y eso se nota en conexiones lentas.
Seguridad y multiplataforma: los datos no salen del servidor, y el mismo servidor atiende a web, móvil y escritorio.	**Coste y complejidad:** servidores, certificados, copias y monitorización (en Azure, el saldo se consume aunque nadie use la máquina).

1.2. Componentes principales

En la Ilustración 1 se representan los componentes de una arquitectura cliente-servidor típica y cómo se relacionan entre sí:

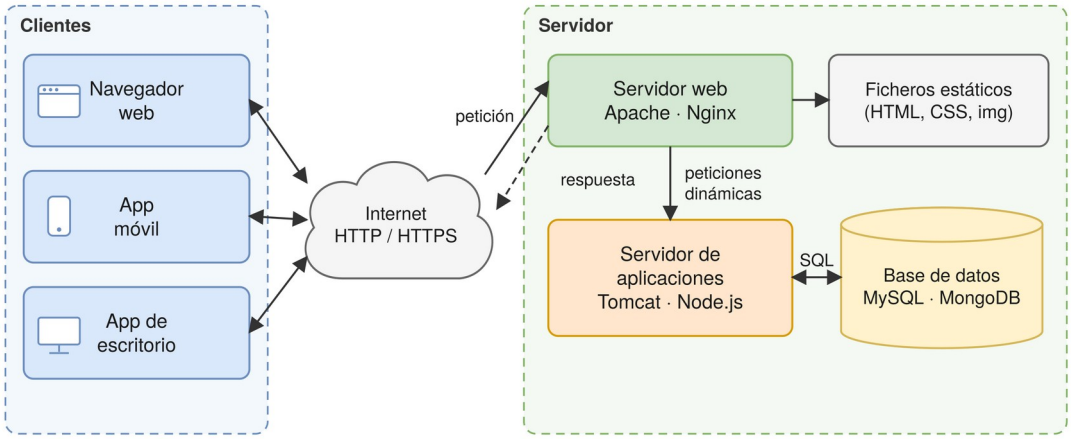


Ilustración 1. Arquitectura cliente-servidor y sus componentes. Fuente: elaboración propia.

Cliente

El cliente es la parte con la que interactúa el usuario. Se encarga de mostrar la información, recoger lo que el usuario hace (clics, formularios, gestos) y transformarlo en peticiones al servidor. Hay tres tipos principales:

- **Navegador web:** el cliente más universal, porque no hay que instalar nada. Interpreta HTML, CSS y JavaScript, y es donde se ejecutan frameworks como Angular o React (Chrome, Firefox, Edge, Safari).
- **Aplicaciones móviles:** nativas (Kotlin, Swift) o multiplataforma (Flutter, React Native), que suelen consumir la misma API que la web y tienen acceso a la cámara, el GPS o las notificaciones.
- **Aplicaciones de escritorio:** programas instalados que también hablan con un servidor, como Spotify o Discord (esta última hecha con tecnologías web por debajo, con Electron). Una aplicación JavaFX que consuma una API REST también entraría aquí.

Servidor

- **Servidor web:** recibe las peticiones HTTP/HTTPS; si se pide un fichero estático lo sirve directamente y, si es algo dinámico, se lo pasa al servidor de aplicaciones. También suele encargarse del cifrado TLS, la compresión o el balanceo. Ejemplos: Apache, Nginx o IIS, ya comparados en prácticas anteriores.
- **Servidor de aplicaciones:** ejecuta la lógica de negocio: valida, aplica las reglas, consulta la base de datos y genera la respuesta. Ejemplos: Tomcat para Java, Node.js o PHP-FPM. Con Spring Boot el Tomcat va embebido en el propio .jar, así que no hace falta instalarlo aparte (Spring, s.f.-a).
- **Base de datos:** guarda la información de forma persistente. Puede ser relacional (MySQL, PostgreSQL), con tablas y SQL, o NoSQL (MongoDB, Cassandra, Redis), pensada para datos menos estructurados o para escalar horizontalmente.

En la práctica no siempre son máquinas distintas: con XAMPP los tres conviven en el mismo equipo, mientras que en producción lo habitual es separarlos, o al menos tenerlos en contenedores distintos con Docker.

1.3. Comunicación entre cliente y servidor

Protocolo HTTP/HTTPS

HTTP (Hypertext Transfer Protocol) es el protocolo de nivel de aplicación con el que se comunican cliente y servidor en la web. Funciona sobre TCP (o sobre QUIC en el caso de HTTP/3) y tiene una característica importante: es un protocolo **sin estado**, es decir, cada petición es independiente y el servidor no "recuerda" las anteriores (Fielding et al., 2022). Para mantener una sesión, por ejemplo que el usuario siga logueado, hay que enviar en cada petición algo que lo identifique, como una cookie o un token.

HTTPS es el mismo HTTP pero cifrado con TLS: lo que viaja (contraseñas, datos de pago, cookies) va cifrado y el certificado permite comprobar que el servidor es quien dice ser. Usa el puerto 443 en lugar del 80 y hoy es prácticamente obligatorio, ya que los navegadores marcan como "no seguro" cualquier sitio en HTTP.

Peticiones y respuestas

Toda la comunicación sigue el mismo esquema: el cliente envía una **petición** y el servidor contesta con una **respuesta**. La petición lleva un método, una ruta, unas cabeceras y, si hace falta, un cuerpo; la respuesta lleva un código de estado, sus cabeceras y el contenido. Por ejemplo, al pedir un libro a una API:

```
GET /api/libros/1 HTTP/1.1
Host: localhost:8080
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 1, "titulo": "Clean Code", "autor": "Robert C. Martin" }
```

Los códigos de estado se agrupan por la primera cifra: **2xx** si todo ha ido bien (200 OK, 201 Created), **3xx** para redirecciones, **4xx** si el error es del cliente (400 Bad Request, 404 Not Found) y **5xx** si el error es del servidor (500 Internal Server Error).

Métodos HTTP

El método indica qué acción se quiere realizar sobre el recurso (MDN Web Docs, s.f.). Hay dos propiedades que conviene tener claras: si el método es **seguro** (no modifica nada en el servidor) y si es **idempotente** (repetir la misma petición varias veces deja el servidor igual que hacerla una sola vez).

Tabla 3. Métodos HTTP más utilizados

Método	Para qué se usa	Seguro	Idempotente	Ejemplo
GET	Obtener un recurso o una lista de recursos	Sí	Sí	GET /api/libros
POST	Crear un recurso nuevo o enviar datos para procesar	No	No	POST /api/libros
PUT	Reemplazar un recurso entero por el que se envía	No	Sí	PUT /api/libros/1
DELETE	Eliminar un recurso	No	Sí	DELETE /api/libros/1
PATCH	Modificar solo una parte de un recurso	No	No *	PATCH /api/libros/1

* PATCH no está definido como idempotente, aunque según cómo se implemente puede llegar a serlo (Dusseault y Snell, 2010).

La diferencia que más confusión da es la de PUT y PATCH: con PUT se envía el recurso completo y se sustituye el que había, mientras que con PATCH solo se mandan los campos que cambian, por ejemplo `{ "disponibles": 0 }`. Y POST no es idempotente porque, si se envía dos veces el mismo formulario, se crean dos registros.

1.4. Ejemplo práctico: consulta de un producto en una tienda online

Para ver todo lo anterior junto, se representa qué ocurre desde que un usuario pulsa en un producto de una tienda online (mitienda.es, inventada) hasta que ve su ficha. Se supone una arquitectura clásica con renderizado en servidor: un servidor web delante, un backend que genera la página y una base de datos relacional.

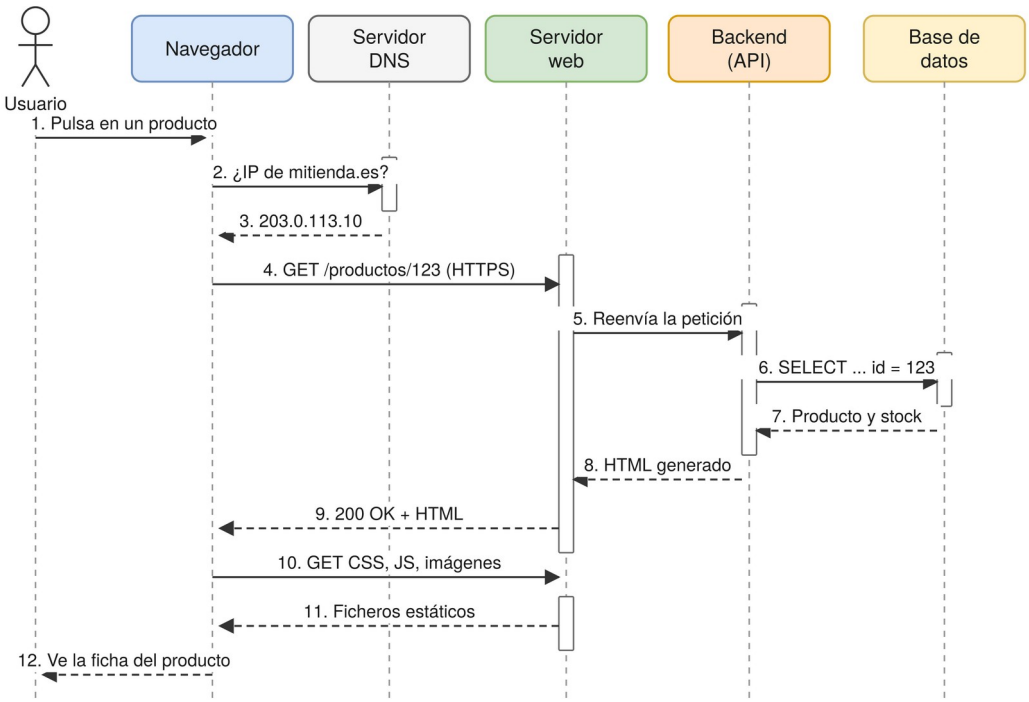


Ilustración 2. Diagrama de secuencia de la consulta de un producto en una tienda online. Fuente: elaboración propia.

Paso a paso:

1. El usuario pulsa en el producto desde el listado (o escribe https://mitienda.es/productos/123).
2. El navegador necesita la IP de mitienda.es y se la pregunta al servidor DNS, si no la tiene ya en caché.
3. El DNS le devuelve la IP del servidor, en este caso 203.0.113.10.
4. El navegador abre una conexión TCP con esa IP en el puerto 443, negocia el cifrado TLS y envía la petición `GET /productos/123`.
5. El servidor web (por ejemplo, Nginx) ve que no se trata de un fichero estático y reenvía la petición al backend, actuando como proxy inverso.
6. El backend, a través de su capa de acceso a datos, lanza la consulta: algo como `SELECT nombre, precio, stock FROM productos WHERE id = 123`.
7. La base de datos devuelve la fila con los datos del producto y el stock disponible.
8. El backend aplica la lógica que corresponda (precio con IVA, si queda stock, si el usuario tiene algún descuento…) y genera el HTML de la página.
9. El servidor web devuelve al navegador una respuesta 200 OK con ese HTML.

10. Al leer el HTML, el navegador encuentra las hojas de estilo, scripts e imágenes, y las pide con nuevas peticiones GET.
11. El servidor web las sirve directamente, sin pasar por el backend (y si la tienda usa una CDN, ni siquiera llegan a su servidor).
12. El navegador construye el DOM, aplica el CSS, ejecuta el JavaScript y el usuario ve la ficha del producto.

Si la tienda fuese una SPA, el flujo cambiaría a partir del paso 4: el navegador ya tendría la aplicación cargada y solo pediría a la API un JSON con los datos (`GET /api/productos/123`), pintando la ficha él mismo. Este caso se explica en el apartado 1.8.

1.5. Modelo de capas

Qué es una arquitectura en capas

Una arquitectura en capas organiza la aplicación en niveles horizontales, cada uno con una responsabilidad concreta, de forma que cada capa solo se comunica con la que tiene inmediatamente debajo y le devuelve el resultado a la de arriba. Es una de las formas más habituales de estructurar una aplicación (Microsoft, s.f.-a). Conviene no confundir **capa** (layer, una división lógica del código) con **nivel** (tier, una división física en máquinas distintas): se pueden tener tres capas dentro del mismo servidor o repartirlas en tres máquinas diferentes.

Objetivos

- Separar responsabilidades, para que la parte visual no tenga que saber SQL y la base de datos no tenga que saber nada de botones.
- Reducir el acoplamiento: cambiar una capa no debería obligar a reescribir las demás.
- Permitir que distintas personas, o equipos, trabajen en paralelo sobre partes diferentes.

Ventajas

- **Mantenimiento más sencillo:** un fallo de validación se busca en la capa de negocio, no en todo el proyecto.
- **Reutilización:** la misma lógica puede servir a una web, a una app móvil o a otra aplicación.
- **Pruebas:** cada capa se puede probar por separado, simulando (mock) la que tiene debajo.
- **Cambio de tecnología acotado:** pasar de MySQL a PostgreSQL debería afectar solo a la capa de datos.

Capas habituales

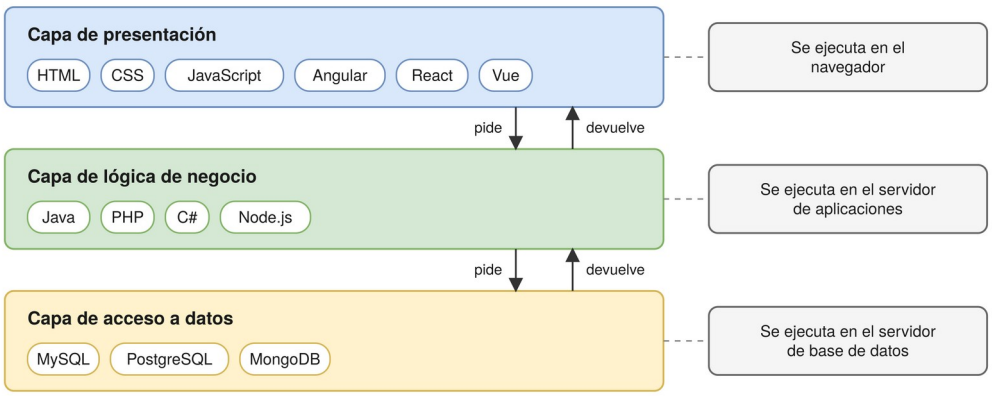


Ilustración 3. Modelo de tres capas con las tecnologías más habituales en cada una. Fuente: elaboración propia.

- **Capa de presentación:** la interfaz, lo que el usuario ve y con lo que interactúa, sin lógica de negocio. Se construye con HTML, CSS y JavaScript y, por encima, con frameworks como Angular, React o Vue, que la organizan en componentes.
- **Capa de lógica de negocio:** las reglas de la aplicación, es decir, qué se puede hacer y con qué condiciones (por ejemplo, que no se pueda prestar un libro sin ejemplares). Se suele desarrollar en Java (Spring), PHP (Laravel, Symfony), C# (ASP.NET) o Node.js (Express, NestJS).
- **Capa de acceso a datos:** lee y guarda la información, aislando al resto de cómo se almacena. En Java es donde están los repositorios (Spring Data JPA o JDBC) y en PHP los modelos de Eloquent o PDO. Por debajo está el gestor: MySQL o PostgreSQL si es relacional, o MongoDB si es documental.

1.6. Modelo-vista-controlador

MVC es un patrón de diseño que divide la aplicación en tres componentes con funciones distintas: modelo, vista y controlador. Se parece al modelo de capas, pero no es lo mismo: las capas organizan toda la aplicación de arriba abajo, mientras que MVC se centra en cómo se atiende cada petición y en separar la interfaz de los datos. De hecho, lo normal es combinarlos: MVC en la parte que recibe las peticiones y capas por debajo.

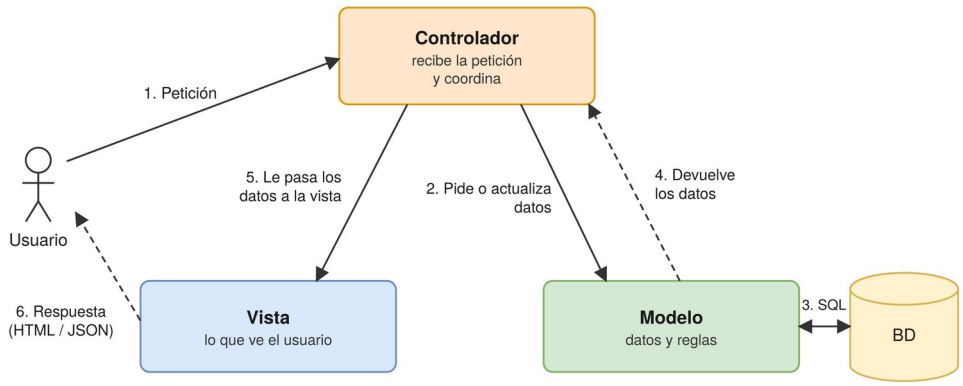


Ilustración 4. Flujo de una petición en el patrón MVC. Fuente: elaboración propia.

Modelo

- **Función:** representa los datos y sus reglas (un libro, un usuario, un pedido).
- **Gestión de datos:** valida que los datos sean correctos y mantiene su estado.
- **Acceso a bases de datos:** es quien habla con la base de datos, directamente o con un ORM (JPA, Eloquent, el ORM de Django). Ni el controlador ni la vista deberían lanzar consultas.

Vista

- **Representación visual:** genera lo que se devuelve al usuario, normalmente HTML a partir de una plantilla (Thymeleaf, Blade, Razor, JSP). En una API REST, la "vista" es el JSON.
- **Interacción con el usuario:** contiene los formularios, enlaces y botones con los que se lanzan nuevas peticiones, sin lógica de negocio.

Controlador

- **Recepción de peticiones:** es el punto de entrada; recibe la petición, lee los parámetros y hace las comprobaciones básicas.
- **Coordinación entre vista y modelo:** pide al modelo los datos (o que los modifique) y elige qué vista devolver con ellos. Hace de director de orquesta, sin hacer él el trabajo pesado.

Frameworks que utilizan MVC

- **Laravel (PHP):** sigue MVC desde la propia estructura de carpetas: modelos Eloquent en app/Models, controladores en app/Http/Controllers y vistas Blade en resources/views (Laravel, s.f.).
- **Spring MVC (Java):** se basa en el patrón Front Controller: un servlet central, el DispatcherServlet, recibe todas las peticiones y las reparte al controlador que toca según las anotaciones @GetMapping, @PostMapping, etc. (Spring, s.f.-b). El modelo suelen ser entidades JPA y servicios, y la vista Thymeleaf o, con @RestController, directamente JSON.
- **ASP.NET Core MVC (C#):** los modelos suelen usar Entity Framework, las vistas se escriben con Razor (HTML mezclado con C#) y el enrutamiento funciona por convención: /Productos/Detalle/5 llama al método Detalle de ProductosController (Microsoft, s.f.-b).
- **Django (Python):** su documentación aclara que no es estrictamente MVC: la vista de MVC es aquí la plantilla (template), y la "view" de Django hace de controlador, decidiendo qué datos se muestran, mientras el sistema de URLs del framework asume el resto. De ahí el nombre MTV, Model-Template-View (Django Software Foundation, s.f.).

1.7. API REST

Qué es REST

REST (Representational State Transfer) es un estilo de arquitectura para sistemas distribuidos que definió Roy Fielding en su tesis doctoral del año 2000. No es un protocolo ni una librería, sino un conjunto de restricciones que, si se cumplen, hacen que el sistema sea escalable, sencillo y fácil de evolucionar (Fielding, 2000). Fielding fue además uno de los autores de la especificación de HTTP, de ahí que REST encaje tan bien con la web.

Qué es una API REST

Una API (Application Programming Interface) es la puerta que expone una aplicación para que otras puedan comunicarse con ella. Una API REST es una API que sigue los principios REST sobre HTTP: cada entidad del sistema es un recurso con su URI (/api/libros, /api/libros/1), se opera sobre él con los métodos HTTP y los datos se intercambian normalmente en JSON: GET /api/libros devuelve la lista (200 OK), POST /api/libros crea un libro (201 Created) y DELETE /api/libros/1 lo elimina (204 No Content). La gracia está en que el mismo backend puede dar servicio a un frontend en Angular, a una app móvil o a otro servidor sin cambiar nada.

Principios REST

- **Cliente-servidor:** separa la interfaz del almacenamiento de los datos. El cliente no sabe cómo guarda los datos el servidor y el servidor no sabe cómo los pinta el cliente, así que cada uno puede evolucionar por su lado.
- **Stateless (sin estado):** cada petición lleva toda la información necesaria para entenderla y el servidor no guarda contexto de sesión; por eso en las APIs se envía el token (un JWT en la cabecera Authorization) en cada llamada. Además facilita escalar, porque cualquier servidor puede atender cualquier petición.
- **Caché:** las respuestas tienen que indicar si se pueden almacenar en caché y durante cuánto tiempo (cabeceras Cache-Control o ETag). Así el cliente, o un intermediario, puede reutilizar una respuesta sin volver a preguntar.
- **Interfaz uniforme:** todos los recursos se manejan igual: se identifican con URIs, se manipulan mediante representaciones (JSON, XML), los mensajes se describen a sí mismos y, en la versión

estricta, las respuestas incluyen enlaces a las acciones posibles (HATEOAS). Para Fielding es la característica central de REST.

- **Recursos identificados mediante URI:** cada recurso tiene una dirección única y estable. Las URIs nombran cosas (sustantivos, en plural) y no acciones: DELETE /api/libros/1, y no /api/borrarLibro?id=1.

La tesis añade otras dos, el sistema en capas y el código bajo demanda (opcional).

1.8. SPA (Single Page Application)

Concepto

Una SPA es una aplicación web que carga una única página HTML y, a partir de ahí, va cambiando el contenido con JavaScript sin recargar el navegador. Cuando el usuario navega, la URL cambia, pero quien gestiona ese cambio es el router del propio framework y no el servidor. Gmail, Google Maps o el tablero de Trello funcionan así.

Funcionamiento

1. El navegador pide la página y recibe un index.html casi vacío (apenas un `<app-root>`) y los ficheros JavaScript de la aplicación.
2. El JavaScript se ejecuta y pinta la interfaz en el navegador (renderizado en el cliente, CSR).
3. Cuando hacen falta datos, la SPA los pide a una API REST con fetch o, en Angular, con HttpClient.
4. Con el JSON recibido actualiza solo la parte de la página que cambia; las siguientes "pantallas" no recargan nada.

La ventaja es una experiencia muy fluida, con el servidor dedicado solo a servir datos; los inconvenientes, una primera carga más lenta y un SEO más complicado, porque el HTML inicial no tiene contenido (Miller y Osmani, 2019).

En la parte izquierda de la Ilustración 5 se representa este funcionamiento, comparado con el SSR que se explica en el siguiente apartado:

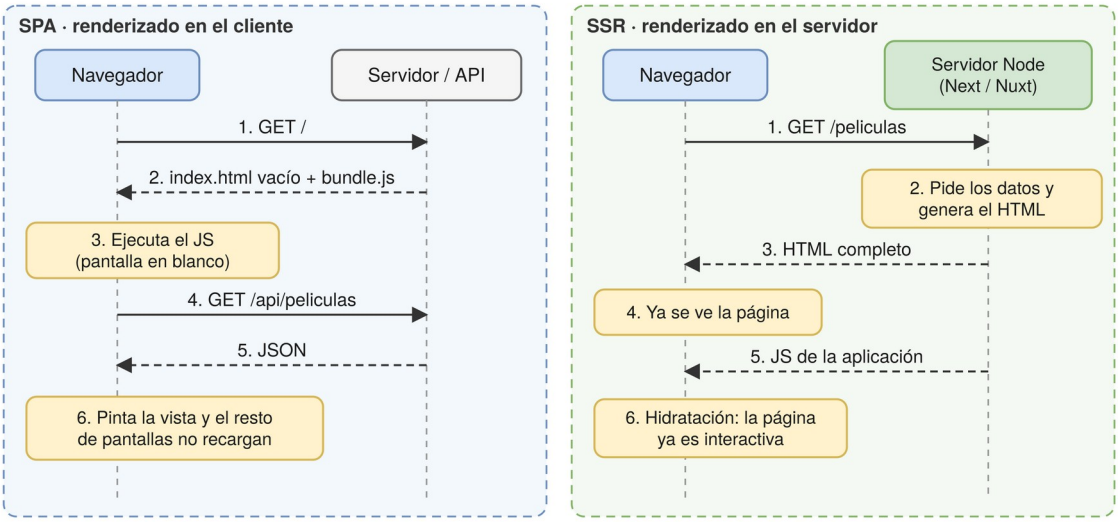


Ilustración 5. Primera carga de una SPA frente a una página con SSR. Fuente: elaboración propia.

Frameworks

- **Angular:** framework completo mantenido por Google, con TypeScript. Trae de serie el router, los formularios, HttpClient y la inyección de dependencias; es más estricto, pero también muy

ordenado, y por eso se usa mucho en empresas (Google, s.f.). Es el que estoy usando ahora en otro proyecto.

- **React:** librería de Meta para construir interfaces con componentes y JSX. Por sí sola solo se ocupa de la vista; el enrutado o la gestión del estado se añaden con otras librerías (React Router, Redux…), lo que da mucha libertad (Meta, s.f.).
- **Vue:** framework progresivo con una curva de aprendizaje más suave. Se puede añadir a una página ya existente o usar para una SPA completa con Vue Router y Pinia (Vue.js, s.f.).

1.9. SSR (Server Side Rendering)

El SSR consiste en generar el HTML de la página en el servidor, en cada petición y con los datos ya incluidos, de forma que el navegador lo recibe listo para mostrar. Es lo que se ha hecho siempre con PHP o JSP, pero los frameworks actuales lo combinan con la SPA: el servidor genera el HTML con los mismos componentes de React o Vue y después el navegador descarga el JavaScript e "hidrata" la página, es decir, le engancha los eventos para que pase a comportarse como una SPA (Miller y Osmani, 2019). Es la parte derecha de la Ilustración 5.

Ventajas

- **Primera carga más rápida en pantalla:** el usuario ve el contenido sin esperar a que se ejecute todo el JavaScript, algo que en móviles poco potentes se nota mucho.
- **SEO:** los buscadores y las previsualizaciones de las redes sociales reciben el HTML con el contenido.

Inconvenientes

- **Más carga en el servidor:** tiene que renderizar en cada petición, lo que cuesta CPU y, por tanto, dinero.
- **Más complejidad:** hay código que se ejecuta en el servidor y en el cliente, y cosas como window o localStorage no existen en el servidor.
- **Tiempos intermedios:** el tiempo hasta el primer byte puede aumentar, y la hidratación tiene su coste: la página ya se ve, pero puede tardar un momento en responder a los clics.

Tabla 4. Comparativa entre SPA y SSR

Aspecto	SPA (CSR)	SSR
Dónde se genera el HTML	En el navegador, con JavaScript	En el servidor, en cada petición
Navegación posterior	Muy fluida, sin recargas	Igual de fluida tras la hidratación
Cuándo usarlo	Paneles internos o aplicaciones con login	Tiendas, noticias, páginas públicas

Frameworks

- **Next.js:** framework sobre React, desarrollado por Vercel. Permite elegir el tipo de renderizado según la página: SSR, generación estática o renderizado en el cliente. En el App Router los componentes son de servidor por defecto (Server Components) y pueden obtener los datos directamente con await (Vercel, s.f.).
- **Nuxt:** el equivalente para Vue. Por defecto usa el llamado renderizado universal (SSR más hidratación), y con routeRules se puede definir el modo de cada ruta; por ejemplo, prerrenderizar la página de inicio y dejar el área de usuario como SPA (Nuxt, s.f.).

1.10. Arquitectura Jamstack

Concepto

Jamstack (por JavaScript, APIs y Markup) es una arquitectura en la que las páginas se generan por adelantado, en el momento del build, y se publican como ficheros estáticos en una CDN. Lo dinámico (formularios, pagos, búsquedas, inicio de sesión) se resuelve desde el navegador llamando a APIs o a funciones serverless. La propia web del proyecto lo resume en dos principios: el prerrenderizado y el desacoplamiento entre frontend y backend (Jamstack, s.f.).

Al no haber un servidor generando páginas en cada petición, es muy rápido (el HTML ya está hecho y se sirve desde el nodo de la CDN más cercano), barato de alojar y con menos superficie de ataque. El punto débil es el contenido que cambia a menudo, ya que cada cambio obliga a regenerar el sitio, aunque existen técnicas intermedias como la regeneración incremental de Next.js. Para generar el sitio se usan Next.js, Astro o Hugo, y para alojarlo Netlify, Vercel o GitHub Pages.

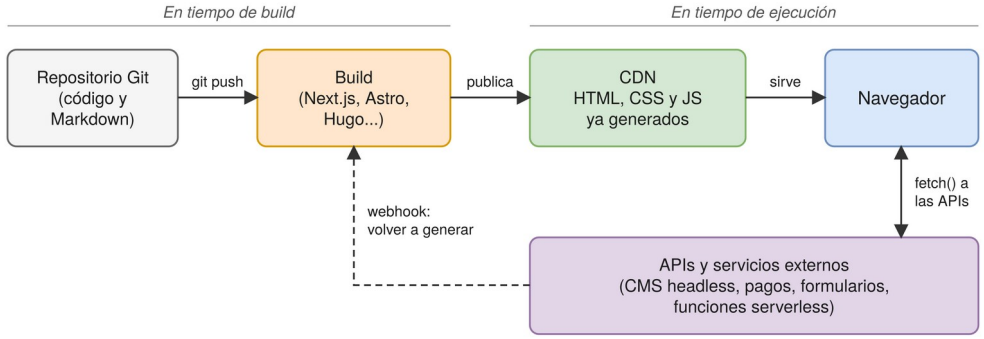


Ilustración 6. Funcionamiento de un sitio Jamstack. Fuente: elaboración propia.

Casos de uso

- Blogs, portfolios y webs corporativas o de marketing.
- Documentación técnica, que cambia poco y tiene que cargar rápido.
- Landing pages y webs de eventos con picos de tráfico puntuales.
- Tiendas pequeñas con un catálogo estable, delegando el pago en un servicio externo como Stripe.

1.11. Frontend

Responsabilidades

- Implementar la interfaz a partir del diseño (Figma, wireframes) y que sea responsive.
- Gestionar la interacción y el estado de la interfaz: navegación, formularios y validaciones en el cliente.
- Consumir la API del backend y mostrar los datos, incluidos los estados de carga y de error.
- Cuidar la accesibilidad, el rendimiento (tamaño del JavaScript, imágenes, carga diferida) y la compatibilidad entre navegadores.

Tecnologías más utilizadas

HTML5, CSS3 (con Sass, Bootstrap o Tailwind), JavaScript y TypeScript; frameworks como Angular, React o Vue; herramientas de construcción como npm, Vite o Webpack; pruebas con Jest, Vitest o Cypress, y Figma para el diseño.

1.12. Backend

Responsabilidades

- Implementar la lógica de negocio y exponerla mediante una API (REST o GraphQL).
- Validar todos los datos que llegan, porque esta es la validación que realmente protege la aplicación.
- Autenticación y autorización (inicio de sesión, roles, tokens JWT) y seguridad en general: inyección SQL, CORS, cifrado de contraseñas.
- Persistencia: modelo de datos, acceso a la base de datos y transacciones.
- Integraciones (pasarelas de pago, correo) y despliegue: caché, logs, monitorización, contenedores y servidores, lo trabajado con Docker y Azure.

Tecnologías más utilizadas

Java con Spring Boot, PHP con Laravel o Symfony, C# con ASP.NET Core, Node.js con Express o NestJS y Python con Django o FastAPI; bases de datos como MySQL, PostgreSQL, MongoDB o Redis; Apache, Nginx o Tomcat como servidores, Docker y nubes como AWS o Azure, y Postman y Swagger para probar y documentar las APIs.

Comparativa

Tabla 5. Comparativa entre frontend y backend

Aspecto	Frontend	Backend
Objetivo	Que el usuario vea la aplicación y pueda usarla de forma clara, rápida y accesible	Procesar la lógica, proteger y guardar los datos, y servírselos al frontend
Dónde se ejecuta	En el navegador o el dispositivo del usuario	En el servidor
Tecnologías	HTML, CSS, JavaScript/TypeScript, Angular, React, Vue, Bootstrap	Java/Spring, PHP/Laravel, C#/.NET, Node.js, Python; MySQL, PostgreSQL, MongoDB
Ejemplos	Formulario de registro con validación en tiempo real; catálogo con filtros; carrito que se actualiza sin recargar	Endpoint POST /api/usuarios que valida y guarda; cálculo del precio final con impuestos; envío del correo de confirmación
Seguridad	No es de fiar: el usuario puede modificar el código	Es donde se aplica la seguridad real

2. CASO DE ESTUDIO: NETFLIX

Se ha elegido Netflix porque es uno de los ejemplos más citados de arquitectura de microservicios y porque la propia empresa publica mucha información técnica, tanto en su blog de ingeniería como en sus repositorios de código abierto. Hay que aclarar que el análisis se basa en esa información pública: Netflix no detalla su arquitectura completa ni su estado actual, así que en algunos puntos se habla de lo que podría estar utilizando.

2.1. Arquitectura cliente-servidor

Netflix es cliente-servidor a gran escala, pero con una particularidad: tiene dos “servidores” muy distintos. Todo lo que ocurre antes de darle a play (iniciar sesión, elegir perfil, navegar por el catálogo, buscar, ver las recomendaciones o gestionar la cuenta) se ejecuta en la nube de AWS. Netflix terminó de migrar sus centros de datos a AWS en enero de 2016, tras unos siete años de trabajo, y aprovechó

para rediseñar casi toda su tecnología en lugar de trasladar tal cual lo que tenía (Izrailevsky et al., 2016).

El vídeo, en cambio, no sale de AWS, sino de **Open Connect**, su propia red de distribución de contenidos: servidores llamados OCA (Open Connect Appliances) que Netflix proporciona gratis a los proveedores de Internet para instalarlos en su red, o que coloca en puntos de intercambio, con el catálogo precargado en horas de poco tráfico (Florance, 2016; Netflix, s.f.-c). Los clientes (televisores, móviles, navegadores, consolas) se comunican con el mismo backend a través de APIs.

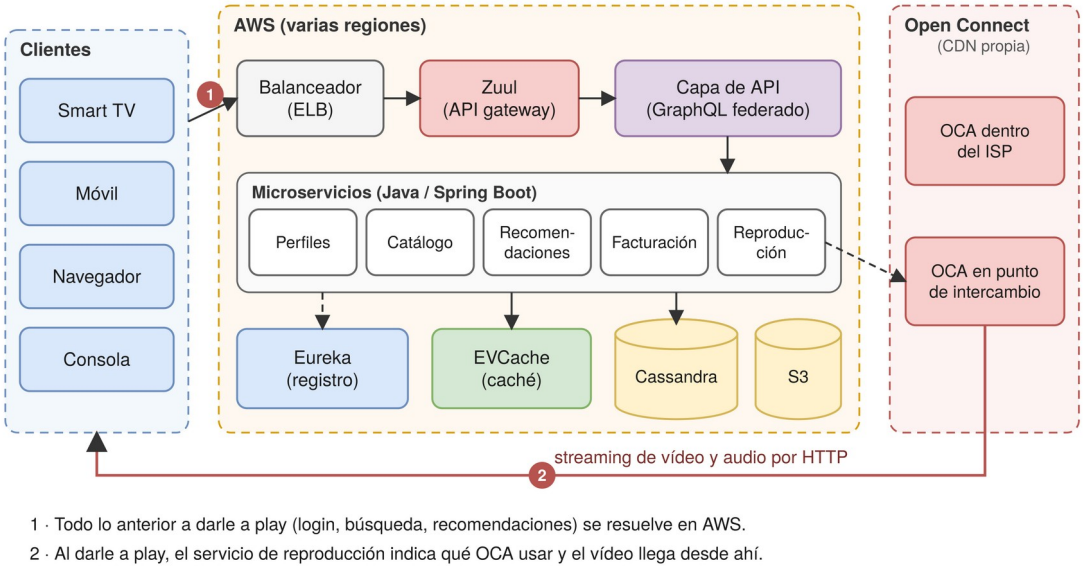


Ilustración 7. Esquema simplificado de la arquitectura de Netflix a partir de su información pública. Fuente: elaboración propia.

2.2. Posible uso de MVC

En conjunto, Netflix no es "una aplicación MVC": es un sistema distribuido formado por cientos de servicios. Pero el patrón sí aparece dentro de cada pieza. Desde 2018, Netflix usa Spring Boot como su framework Java principal (Netflix Technology Blog, 2018), y un microservicio con Spring Boot se organiza normalmente con controladores que reciben las peticiones, servicios con la lógica y modelos de datos, exactamente como en Spring MVC; la vista, en este caso, es la respuesta en JSON o GraphQL.

En la web, la interfaz se construye con React. Según un caso de estudio publicado con su equipo, las páginas se renderizan con React en el servidor, y en la página de inicio sin sesión llegaron a quitar React del cliente, reduciendo a la mitad el tiempo hasta que es interactiva (Osmani, 2018): es decir, combinan SSR con el comportamiento de una SPA.

2.3. APIs utilizadas

- **Zuul (API gateway):** es la puerta de entrada de todo el tráfico. Enruta cada petición al servicio que corresponde y le aplica filtros (autenticación, enrutado, métricas). En 2016 lo rehicieron sobre Netty para que fuera asíncrono y no bloqueante, y así poder mantener muchas más conexiones abiertas a la vez (Netflix Technology Blog, 2016).
- **Capa de API con GraphQL federado:** en lugar de que cada aplicación hable con cientos de microservicios, una capa agrega los datos. Netflix la ha ido migrando a GraphQL con federación: cada equipo publica su parte del esquema y una pasarela la une en una única API. Empezaron por la API de Studio, la de producción de contenidos (Netflix Technology Blog, 2020), y crearon para ello el framework DGS, basado en Spring Boot.

- **Eureka (registro de servicios):** los microservicios se registran en Eureka, un servicio REST de código abierto creado por Netflix, para poder localizarse entre sí y repartir la carga (Netflix, s.f.-a).
- **API de reproducción y Open Connect:** al darle a play, la aplicación pide al servicio de reproducción qué servidor usar; el sistema de control, en AWS, elige el OCA más adecuado y el cliente descarga el vídeo directamente de él por HTTP.

2.4. Frontend y backend

El **frontend** lo forman las aplicaciones de cada plataforma (la web con React, las apps de Android e iOS y las de televisores y consolas) junto con el reproductor, que además adapta la calidad del vídeo a la conexión. El **backend** está en AWS: balanceador de carga, Zuul, la capa de API y los microservicios de perfiles, catálogo, recomendaciones, facturación o reproducción, con sus bases de datos (Cassandra) y cachés (EVCache, que funciona sobre memcached; Netflix, s.f.-b). Open Connect quedaría como una parte aparte, dedicada solo a entregar el vídeo.

2.5. Uso de microservicios

Es el aspecto por el que Netflix es más conocido. En 2008 sufrió una corrupción grave de su base de datos que le impidió enviar DVDs durante tres días, y a raíz de eso decidió abandonar los sistemas con un único punto de fallo y pasar a sistemas distribuidos y escalables horizontalmente en la nube (Izrailevsky et al., 2016). Hoy su backend se compone de cientos de microservicios independientes, cada uno con su equipo, que se despliegan y escalan por separado, que es la idea de microservicios tal y como la describen Lewis y Fowler (2014).

Las ventajas son claras a esa escala: si falla el servicio de recomendaciones se puede seguir viendo una serie, y cada equipo despliega sin coordinarse con todos los demás. El coste es que hace falta un registro de servicios, un gateway, mucha monitorización y asumir que los fallos parciales son lo normal; de ahí su costumbre de provocar fallos a propósito en producción con la Simian Army.

2.6. Tecnologías que podrían emplearse

Tabla 6. Tecnologías de Netflix por capa, según su información pública

Capa	Tecnologías	Comentario
Clientes	React (web), apps nativas Android e iOS, apps de TV y consolas	React y su SSR constan en sus artículos; el resto, según plataforma
Infraestructura	AWS en varias regiones, balanceadores de carga	Toda la parte anterior al play
Gateway	Zuul 2 (Netty)	Proyecto de código abierto de Netflix
Capa de API	GraphQL federado, framework DGS	Una única API para los clientes
Microservicios	Java con Spring Boot	Framework principal desde 2018
Registro de servicios	Eureka	Descubrimiento y balanceo entre servicios
Datos y caché	Cassandra, EVCache (basado en memcached), Amazon S3	BD distribuida, caché en memoria y almacenamiento
Entrega de vídeo	Open Connect (OCA)	CDN propia dentro de los ISP y en puntos de intercambio

3. DIAGRAMA DE UNA APLICACIÓN WEB PROPIA: BIBLIOTECA DECROLY

La Biblioteca Decroly es una API REST desarrollada con Spring Boot 4 y Java 25 para gestionar el catálogo de una biblioteca escolar: libros y categorías. Ahora mismo guarda los datos en memoria (listas dentro de los servicios, con unos datos de ejemplo cargados al arrancar) y no tiene frontend propio; se prueba con Postman, con una colección que tiene una petición por cada ruta, y con Swagger UI, que se genera automáticamente con springdoc-openapi. En el diagrama se distingue lo que ya está implementado (línea continua) de la ampliación propuesta (línea discontinua): un frontend en Angular servido por Nginx y una base de datos MySQL con Spring Data JPA.

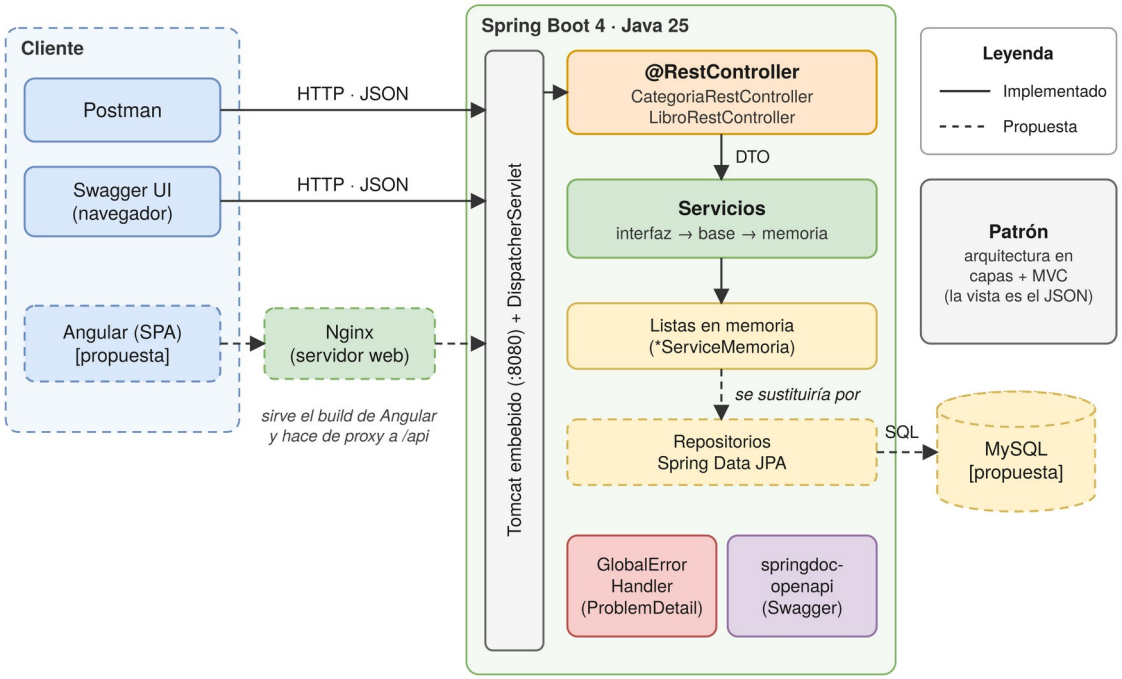


Ilustración 8. Arquitectura de la Biblioteca Decroly: estado actual y ampliación propuesta. Fuente: elaboración propia.

3.1. Componentes

- **Cliente:** actualmente, Postman y Swagger UI (en /swagger-ui.html). Como ampliación, una SPA en Angular que consuma la API con HttpClient.
- **Servidor web:** hoy es el Tomcat que Spring Boot lleva embebido, en el puerto 8080. En la propuesta, Nginx serviría los ficheros compilados de Angular (ng build) y haría de proxy inverso de /api hacia Spring Boot, de modo que todo sale por el mismo dominio y se evitan los problemas de CORS.
- **Backend:** las peticiones entran por el DispatcherServlet de Spring MVC, que las pasa a los controladores (CategoriaRestController y LibroRestController). Estos reciben los DTO del cuerpo y llaman a los servicios, que siguen la estructura interfaz → clase base → implementación en memoria. Los errores se centralizan en un GlobalErrorHandler que responde en formato ProblemDetail.
- **Base de datos:** de momento no hay, y las listas se pierden al reiniciar. La propuesta es MySQL con repositorios de Spring Data JPA (por ejemplo, `LibroRepository extends JpaRepository<Libro, Long>`) que sustituirían a las implementaciones en memoria.
- **API REST:** expone dos recursos, libros y categorías, con las rutas de la siguiente tabla.

Tabla 7. Endpoints de la API REST de la Biblioteca Decroly

Método	Ruta	Descripción
GET	/api/categorias	Lista todas las categorías
GET	/api/categorias/{id}	Obtiene una categoría por su id
POST	/api/categorias	Crea una categoría
PUT	/api/categorias/{id}	Modifica una categoría
DELETE	/api/categorias/{id}	Elimina una categoría
GET	/api/libros	Lista todos los libros
GET	/api/libros/disponibles	Lista los libros con ejemplares disponibles
GET	/api/libros/autor/{autor}	Lista los libros de un autor
GET	/api/libros/{id}/libros	Lista los libros de una categoría (id de la categoría)
GET	/api/libros/{id}	Obtiene un libro por su id
POST	/api/libros	Crea un libro
PUT	/api/libros/{id}	Modifica un libro
DELETE	/api/libros/{id}	Elimina un libro

3.2. Patrón arquitectónico utilizado

La aplicación sigue una **arquitectura en capas** (controlador → servicio → almacenamiento) combinada con **MVC** en la versión REST de Spring: el controlador es el @RestController, el modelo lo forman las entidades Libro y Categoria junto con los servicios, y la vista es la representación en JSON que genera Spring con Jackson. Con la ampliación, el conjunto pasaría a ser una arquitectura cliente-servidor de tres niveles: Angular en el navegador, Spring Boot en el servidor de aplicaciones y MySQL como servidor de datos.

3.3. Justificación de las decisiones tomadas

- **Spring Boot:** configura casi todo por sí mismo (Tomcat embebido, JSON, validación) y es un estándar en empresa, como se ha visto con Netflix.
- **API REST con JSON:** separa el backend de cualquier cliente; la misma API vale para Angular, para una app móvil o para Postman.
- **Capas e interfaces en los servicios:** permiten cambiar el almacenamiento (de memoria a JPA con MySQL) sin tocar los controladores, que es justo lo que plantea la ampliación, y probar los servicios con JUnit.
- **DTO para las peticiones:** evitan exponer directamente las entidades y controlan qué campos puede enviar el cliente; el id, por ejemplo, no lo decide él.
- **Errores centralizados con ProblemDetail:** todas las respuestas de error tienen el mismo formato (400 si faltan datos obligatorios, 404 si no existe el recurso), lo que facilita mucho mostrarlas en el frontend.
- **Swagger/OpenAPI:** la documentación sale del propio código, así que no se queda desfasada.
- **Angular:** con TypeScript, los modelos del frontend pueden reflejar los DTO del backend, y trae de serie HttpClient y el router.
- **MySQL con Spring Data JPA:** los datos son claramente relacionales (un libro pertenece a una categoría), es la base de datos que se usa en el ciclo y se levanta fácilmente en un contenedor Docker.
- **Nginx:** más ligero que Apache para servir estáticos y hacer de proxy, como se vio en la práctica de Docker.

- **Monolito y no microservicios:** con dos recursos, separar la aplicación en microservicios sería complicarla sin ningún motivo. Lo que tiene sentido para Netflix, con cientos de equipos, no lo tiene aquí.

CONCLUSIONES

Después de hacer el trabajo, lo que más claro me queda es que casi todo lo visto se apoya en la misma idea: separar responsabilidades. La arquitectura cliente-servidor separa a quien pide de quien tiene los datos; las capas separan la presentación, la lógica y los datos; MVC separa la entrada de la petición, los datos y lo que se muestra, y REST separa el backend de cualquier cliente que quiera usarlo. Cambia el nivel al que se aplica, pero el objetivo es siempre poder cambiar una parte sin romper las demás.

También me ha quedado claro que no hay una opción mejor para todo: para un panel interno con login tiene sentido una SPA; para una tienda en la que importan el SEO y la primera carga, SSR; y para un blog o una documentación, Jamstack. Lo importante es elegir según el proyecto y no por moda, y Netflix lo demuestra, porque mezcla SSR y SPA en la misma web según le conviene.

Con los microservicios pasa algo parecido: en Netflix tienen todo el sentido por su tamaño, pero traen mucha complejidad extra (gateway, registro de servicios, monitorización…). Para un proyecto como la Biblioteca Decroly, un monolito bien organizado en capas es más que suficiente y, si está bien diseñado desde el principio, crecer después resulta mucho más sencillo.

Por último, hacer el diagrama de mi propia aplicación me ha servido para ver que la estructura que seguimos con Spring Boot (controladores, servicios con interfaces, DTO, manejo centralizado de errores) no es un capricho: es lo que permite añadir una base de datos o un frontend sin rehacer el proyecto. Y aunque frontend y backend se trabajen por separado, conviene entender los dos, porque decisiones como dónde validar los datos o cómo diseñar la API afectan a ambos lados.

BIBLIOGRAFÍA

Django Software Foundation. (s.f.). *FAQ: General*. Obtenido de https://docs.djangoproject.com/en/stable/faq/general/

Dusseault, L. y Snell, J. (2010). *RFC 5789: PATCH Method for HTTP*. IETF. Obtenido de https://www.rfc-editor.org/rfc/rfc5789

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral, University of California, Irvine]. Obtenido de https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M. y Reschke, J. (2022). *RFC 9110: HTTP Semantics*. IETF. Obtenido de https://www.rfc-editor.org/rfc/rfc9110

Florance, K. (2016). *How Netflix works with ISPs around the globe to deliver a great viewing experience*. Netflix. Obtenido de https://about.netflix.com/en/news/how-netflix-works-with-isps-around-the-globe-to-deliver-a-great-viewing-experience

Google. (s.f.). *Angular*. Obtenido de https://angular.dev

Izrailevsky, Y., Vlaovic, S. y Meshenberg, R. (2016). *Completing the Netflix cloud migration*. Netflix. Obtenido de https://about.netflix.com/en/news/completing-the-netflix-cloud-migration

Jamstack. (s.f.). *What is Jamstack?* Obtenido de https://jamstack.org/what-is-jamstack/

Laravel. (s.f.). *Laravel documentation*. Obtenido de https://laravel.com/docs

Lewis, J. y Fowler, M. (2014). *Microservices*. martinfowler.com. Obtenido de https://martinfowler.com/articles/microservices.html

MDN Web Docs. (s.f.). *Métodos de petición HTTP*. Mozilla. Obtenido de https://developer.mozilla.org/es/docs/Web/HTTP/Methods

Meta. (s.f.). *React*. Obtenido de https://react.dev

Microsoft. (s.f.-a). *Arquitecturas comunes de aplicaciones web*. Microsoft Learn. Obtenido de https://learn.microsoft.com/es-es/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures

Microsoft. (s.f.-b). *Información general de ASP.NET Core MVC*. Microsoft Learn. Obtenido de https://learn.microsoft.com/es-es/aspnet/core/mvc/overview

Miller, J. y Osmani, A. (2019). *Rendering on the Web*. web.dev. Obtenido de https://web.dev/articles/rendering-on-the-web

Netflix. (s.f.-a). *Eureka* [Repositorio de código]. GitHub. Obtenido de https://github.com/Netflix/eureka

Netflix. (s.f.-b). *EVCache* [Repositorio de código]. GitHub. Obtenido de https://github.com/Netflix/EVCache

Netflix. (s.f.-c). *Open Connect Overview*. Obtenido de https://openconnect.netflix.com/Open-Connect-Overview.pdf

Netflix Technology Blog. (2016). *Zuul 2: The Netflix journey to asynchronous, non-blocking systems*. Obtenido de https://netflixtechblog.com/zuul-2-the-netflix-journey-to-asynchronous-non-blocking-systems-45947377fb5c

Netflix Technology Blog. (2018). *Netflix OSS and Spring Boot: Coming full circle*. Obtenido de https://netflixtechblog.com/netflix-oss-and-spring-boot-coming-full-circle-4855947713a0

Netflix Technology Blog. (2020). *How Netflix scales its API with GraphQL federation (Part 1)*. Obtenido de https://netflixtechblog.com/how-netflix-scales-its-api-with-graphql-federation-part-1-ae3557c187e2

Nuxt. (s.f.). *Rendering modes*. Obtenido de https://nuxt.com/docs/guide/concepts/rendering

Osmani, A. (2018). *A Netflix web performance case study*. Chrome Developers. Obtenido de https://medium.com/dev-channel/a-netflix-web-performance-case-study-c0bcde26a9d9

Spring. (s.f.-a). *Spring Boot reference documentation*. Obtenido de https://docs.spring.io/spring-boot/index.html

Spring. (s.f.-b). *Spring Web MVC: DispatcherServlet*. Obtenido de https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-servlet.html

Vercel. (s.f.). *Next.js documentation*. Obtenido de https://nextjs.org/docs

Vue.js. (s.f.). *Vue.js*. Obtenido de https://vuejs.org